

DocProcessor

Convierte la documentación en un mapa de características verificable para la automatización de QA.

Resumen

DocProcessor es un módulo Go independiente y completamente desacoplado que carga la documentación del proyecto, construye mapas estructurados de características y rastrea la cobertura de verificación. Está diseñado para funcionar con agentes LLM para la extracción inteligente de características, pero también incluye extracción heurística para uso sin conexión.

Descripción breve

Un módulo Go agnóstico al proyecto para el procesamiento de documentación y la extracción de mapas de características. Analiza los documentos y los convierte en mapas estructurados de características, registrando qué funcionalidades han sido verificadas —ya sea mediante agentes LLM para extracción inteligente o heurísticas sin conexión—, y alimenta la automatización de QA con una garantía anti-engaño que siempre coincide con la realidad.

Descripción detallada

Todos los equipos de desarrollo conviven con la misma mentira lenta: la documentación promete funcionalidades, las pruebas cubren algo parecido y nadie puede afirmar con certeza si ambas describen el mismo producto. DocProcessor existe para hacer visible y medible esa brecha. A partir de la documentación de un proyecto, construye un mapa estructurado de características —un modelo enumerado y legible por máquina de todo lo que el producto afirma hacer— y rastrea la cobertura de verificación frente a él, de modo que la pregunta “*¿esta funcionalidad documentada está realmente probada?*” deja de ser un debate de pasillo y se convierte en una consulta con respuesta. Opera en modo dual: utiliza agentes LLM para una extracción semántica e

inteligente de características cuando están disponibles, y recurre a un analizador heurístico para uso completamente sin conexión, evitando así depender de un modelo y funcionando igual en un entorno de CI aislado o en el portátil de un desarrollador con modo avión.

Desde el punto de vista arquitectónico, es un módulo Go independiente, agnóstico al proyecto y completamente desacoplado (CONST-051(B)): no incluye valores específicos de ningún proyecto y se integra como un submódulo de código equivalente, permitiendo que cualquier proyecto lo adopte sin heredar suposiciones ajenas. Además, se somete a los mismos estándares que impone: sus propias afirmaciones están sujetas al pacto anti-engaño (CONST-035) y a las reglas de cobertura de automatización total (CONST-048), lo que significa que cada capacidad anunciada en su README es validada por una prueba automatizada o un script *Challenge* que confirma un comportamiento real y utilizable por el usuario final, en lugar de limitarse a finalizar sin errores. Las cadenas de texto orientadas al usuario pasan por el traductor de internacionalización CONST-046. El objetivo de todo esto es cerrar el ciclo: DocProcessor es el lado de entrada del ciclo de QA que HelixQA completa —extrae el mapa de características de la documentación, HelixQA demuestra cada funcionalidad mapeada con evidencia de ejecución en tiempo real, y la documentación, las pruebas y el comportamiento entregado se ven obligados a converger en lugar de divergir silenciosamente versión tras versión.

Por qué lo creamos

La documentación y las pruebas se distancian: los documentos prometen funcionalidades que ninguna prueba valida, y el QA no puede determinar fácilmente qué significa “completo”. DocProcessor transforma la documentación en un mapa de características legible por máquina para que la cobertura de verificación pueda medirse frente a lo que realmente se prometió.

Por qué es un cambio de juego

Convierte la pregunta más difusa en la entrega de software —“¿lo que entregamos coincide con lo que dijimos que entregaríamos?”— en algo automatizable y verificable de forma continua, y lo hace sin una dependencia estricta de AI: extracción mediante LLM cuando hay un modelo disponible, o heurísticas cuando no lo hay, de modo que la misma garantía se mantiene en cualquier entorno, desde un ejecutor sin conexión hasta una canalización totalmente autónoma.

Qué tiene de innovador

- Extracción de mapas de características a partir de la documentación con seguimiento de cobertura de verificación.
- Extracción dual: impulsada por agentes LLM o heurística/fuera de línea.
- Desacoplamiento agnóstico al proyecto y sin configuración (CONST-051(B)).
- Autoverificación anti-engaño: las afirmaciones del README están respaldadas por pruebas/Desafíos (CONST-035/048).

Desafíos y soluciones

- **Operación opcional del modelo:** resuelto con un mecanismo de extracción heurística como alternativa para que el módulo funcione sin conexión.
- **Mantener alineados la documentación y la realidad:** resuelto mediante mapas estructurados de características y seguimiento de cobertura de verificación, integrados en el ciclo de control de calidad.
- **Reutilización:** resuelto mediante un desacoplamiento estricto y el consumo de submódulos en una misma base de código.
- **Credibilidad de sus propias afirmaciones:** resuelto con pruebas/Desafíos anti-engaño para cada capacidad anunciada.

Tecnologías empleadas (por qué y cómo)

- **Go (1.25+)** — núcleo del módulo; licencia Apache-2.0.
- **Agentes LLM** — extracción semántica inteligente de características (opcional).
- **Analizador heurístico** — alternativa de extracción de características sin conexión.
- **Traductor i18n (pkg/i18n)** — cadenas localizadas según CONST-046.
- **Marco de Desafíos** — verificación anti-engaño de las afirmaciones del propio módulo.